Table des matières

Exercices corrigés 03 — Charger des fichiers dans la couche RAW 1
Questions courtes 1
Questions pratiques 2
Diagnostics 2
Questions à développer 3
Mises en situation 3
Corrigé 4

Exercices corrigés 03 — Charger des fichiers dans la couche RAW

Comment travailler ces exercices. Répondez seul, puis vérifiez avec le corrigé. Les exercices marqués [labo] portent sur la `RAW` que vous venez de charger ; les autres vérifient que le cours est clair dans votre tête. Tout vient de la Théorie 03 et du Labo 03.

Le corrigé suit les énoncés, en fin de document. Répondez à l'énoncé en entier avant d'y passer : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. Les trois voies. Nommez les trois façons d'extraire vues au cours, avec la source de Boréal qui sert d'exemple à chacune.
2. Complet ou incrémental. Donnez un avantage et un inconvénient de chaque forme de chargement.
3. Un extrait n'est pas la base. Pourquoi les extraits de l'ERP comptent-ils 26 538 en-têtes de commande alors que la base en compte 25 483 ?
4. Les données maîtres. Les extraits livrent `KNA1` en entier chaque mois. Pourquoi la `RAW` en garde-t-elle 7 481 lignes plutôt que 156 ?
5. Idempotence. Définissez-la en une phrase, et dites pourquoi un pipeline en a besoin plus qu'un script lancé une seule fois.
6. Trois stratégies. Remplacer, fusionner, tenir un journal : donnez un cas de Boréal pour chacune.
7. L'empreinte. À quoi sert le SHA-256 dans le journal des chargements, et que se passe-t-il quand un fichier revient avec la même empreinte ?
8. Le fichier corrigé. Un export complet revient sous le même nom, avec un contenu différent. Que fait le chargeur, étape par étape ?

© 2026 iAlexMG. Tous droits réservés.

9. La transaction. Pourquoi le chargement d'un fichier et sa ligne au journal s'écrivent-ils dans la même transaction ?
10. Le lot. Que vaut `_lot` pour un extrait de l'ERP, pour un jour de télémétrie et pour la GMAO ?
11. Trois colonnes de lignage. Nommez-les et donnez, pour chacune, une question à laquelle elle permet de répondre.
12. Tout en texte. Donnez deux colonnes de Boréal qu'un type deviné abîmerait, et dites ce qu'elles deviennent dans la `RAW`.
13. Le champ vide. Que devient un champ vide d'un CSV dans la `RAW`, et pourquoi ce choix plutôt que la chaîne vide ?
14. Bronze et `RAW`. Les deux gardent le brut. Qu'est-ce qui les distingue, et où vit la télémétrie ?
15. [labo] Le second passage. Qu'affiche le chargeur quand on le relance sans rien changer, et pourquoi ?
16. [labo] Le verrou. Que répond le chargeur si l'interface de DuckDB tient encore l'entrepôt, et pourquoi cette règle existe.
17. [labo] Le corpus. Pourquoi les documents se déposent-ils tels quels dans le bronze au lieu d'entrer dans des tables, et que devient le catalogue ?

Questions pratiques

Ici on écrit vraiment : des requêtes, des commandes. Tapez votre réponse avant de la comparer. La `RAW` doit être chargée. Les requêtes se tapent dans un carnet de l'interface web, lancée depuis `Projet_Boreal`, après l'`ATTACH` en lecture seule du labo (3.1), et nomment les tables en entier, comme au labo : `entrepot.raw.erp_vbak`, `entrepot.pipeline.chargements`. À la fin, `DETACH entrepot;`, avant tout script qui écrit.

P1. Le journal d'un mois. Écrivez la requête qui donne, pour le lot `2024-05`, la liste des fichiers chargés avec leur nombre de lignes, triée par table. Combien de fichiers, et combien de lignes en tout ?

P2. Les lots d'une table. Écrivez la requête qui compte les lots de `raw.garanties_clients`, donne le premier et le dernier, et le nombre de lignes du lot `2025-09`.

P3. L'état courant des clients. Écrivez la requête qui garde, pour chaque `client_id`, sa version du lot le plus récent, et qui compte les clients obtenus. Comparez ce nombre au nombre de lignes de la table, et expliquez l'écart.

P4. Le doublon de mars 2023. Écrivez la requête qui prouve que les lignes du lot `2023-03_bis` sont exactement celles du lot `2023-03`, colonnes de lignage exclues.

P5. Les doublons de la télémétrie. Écrivez la requête qui compte les identifiants de message reçus plus d'une fois, et le nombre de messages en trop. Retrouvez-vous les 2 303 doublons annoncés par le générateur ?

P6. Les tables les plus lourdes. Écrivez la requête qui donne les cinq tables de la `RAW` qui comptent le plus de lignes, d'après le journal des chargements.

Diagnostics

© 2026 iAlexMG. Tous droits réservés.

On vous montre quelque chose qui ne va pas. Dites ce qui cloche, pourquoi, et donnez le correctif. Ces situations sont toutes arrivées à quelqu'un.

E1. Un collègue charge les extraits sans journal, avec un `INSERT` par fichier lancé depuis un script qu'il relance quand il a un doute. `raw.erp_vbak` compte 53 076 lignes pour 25 483 commandes, et les rapports de ventes donnent le double du chiffre d'affaires réel.

E2. Une collègue trouve que le journal devrait reconnaître un fichier à son empreinte plutôt qu'à son chemin, pour attraper `2023-03_bis`. Elle change le chargeur en ce sens. Le chargement passe, mais `raw.erp_t005u` ne compte plus que 20 lignes, toutes au lot `2022-01`, et l'ingénieur analytique signale que sa table des régions est vide quand il filtre sur le dernier lot.

Questions à développer

Ces questions ne se répondent pas en un mot : on veut un court paragraphe de raisonnement. Rédigez votre réponse avant d'ouvrir le corrigé.

A. Défendre le doublon. Le responsable des ventes découvre que la `RAW` contient deux fois les commandes de mars 2023 et demande qu'on les retire « à la source ». Répondez-lui : ce que la `RAW` garantit, ce qui arrive si on la corrige, et qui règle le problème, où.

Mises en situation

Le format le plus proche du travail réel. On vous décrit un organisme, ses données et ses contraintes ; à vous de décider et de justifier. Il n'y a pas une seule bonne réponse, mais il y a des réponses indéfendables. Le corrigé donne une solution défendable et dit ce qui serait acceptable aussi.

M1. Coopérative forestière des Hautes-Terres. Chaque nuit, six scieries déposent sur un serveur de fichiers un export de leur production de la veille, nommé `production_<scierie>_AAAA-MM-JJ.csv`. Il arrive qu'une scierie redépose un fichier corrigé le lendemain, sous le même nom, et qu'une autre dépose deux fois le même fichier sous deux noms, avec un suffixe `_v2`. Les analystes veulent l'état juste de chaque journée, et la direction veut savoir ce qui a été corrigé.

Décrivez le chargement : la clé du journal, la règle du lot, ce qui entre dans la `RAW`, et le contrôle qui signale les redépôts.

© 2026 iAlexMG. Tous droits réservés.

Corrigé

La numérotation est celle des énoncés. Les chiffres sont ceux de la génération par défaut de Boréal, chargée par `charger_raw.py` ; les requêtes se lancent depuis `Projet_Boreal`.

Questions courtes

1. Un dépôt de fichiers : les extraits mensuels de l'ERP et du portail des garanties, l'archive de la télémétrie, les exports de l'usine. Une base, par un pilote : la base de l'ERP, dans PostgreSQL, lue par ODBC. Une API REST : les observations d'Environnement et Changement climatique Canada.

Théorie : `Theorie_03_DE.md`, section 1.

2. Complet : simple et sûr, l'état obtenu est juste, mais il coûte cher dès que la table grossit et il efface la version de la veille. Incrémental : peu coûteux et il garde tout, mais il exige de savoir ce qui est déjà chargé, sans quoi il double.

Théorie : `Theorie_03_DE.md`, section 2.

3. Parce que les extraits sont incrémentaux : une commande modifiée le mois suivant sa création revient dans l'extrait suivant, dans sa nouvelle version. Les 26 538 lignes portent donc 25 483 commandes, dont certaines en plusieurs versions. L'état de la base se retrouve en gardant, pour chaque commande, la ligne du lot le plus récent.

Théorie : `Theorie_03_DE.md`, section 2.

4. Parce que la `RAW` garde chaque livraison plutôt que de remplacer. Les 49 fichiers `KNA1` livrent chacun le fichier complet des concessionnaires, de 150 à 156 selon le mois : 7 481 lignes en tout, pour 156 concessionnaires distincts. C'est la mémoire de chaque ouverture et de chaque fermeture, que le remplacement aurait effacée.

Théorie : `Theorie_03_DE.md`, sections 2 et 4.

5. Un traitement est idempotent quand le relancer donne le même résultat que le lancer une fois. Un pipeline se relance tout le temps : après une panne au milieu du travail, quand l'orchestrateur rattrape une nuit manquée, quand quelqu'un veut simplement vérifier. Un script lancé une seule fois, sous surveillance, n'a pas ce besoin.

Théorie : `Theorie_03_DE.md`, section 3.

6. Remplacer : le registre des actifs, la GMAO, le catalogue du corpus, livrés en entier à chaque fois. Fusionner : une source à clé fiable dont on ne veut que l'état courant, comme une table de stocks. Tenir un journal : les extraits mensuels de l'ERP et des garanties, et les jours de télémétrie, où l'on veut garder chaque version livrée.

Théorie : `Theorie_03_DE.md`, section 3.

© 2026 iAlexMG. Tous droits réservés.

7. Le SHA-256 résume le contenu d'un fichier : deux contenus identiques ont la même empreinte, un seul octet changé en donne une autre. Un fichier déjà au journal avec la même empreinte est sauté, sans être relu.

Théorie : `Theorie_03_DE.md`, section 3.

8. Le journal connaît le chemin, mais avec une autre empreinte. Le chargeur retire les lignes de l'ancienne version, charge la nouvelle, et note la nouvelle empreinte, le tout dans la même transaction. C'est ainsi qu'un export complet se met à jour.

Théorie : `Theorie_03_DE.md`, section 3.

9. Pour qu'une panne au milieu du travail ne laisse pas une moitié de livraison. Sans la transaction, la table pourrait contenir des lignes qu'aucune ligne de journal ne réclame, ou le journal annoncer un fichier dont les lignes manquent. Avec elle, le passage suivant reprend proprement.

Théorie : `Theorie_03_DE.md`, section 3.

10. Pour un extrait de l'ERP, le nom du dossier du mois : `2024-05`, et `2023-03_bis` pour l'extrait livré deux fois. Pour un jour de télémétrie, la date tirée du nom du fichier : `2024-05-03`. Pour la GMAO, livrée en entier, `complet`.

Labo : `Labo_03_DE.md`, étape 2.

11. `_fichier` dit d'où vient la ligne : où retourner quand une valeur surprend, et quoi rejouer. `_lot` dit de quelle livraison elle vient : quel mois filtrer, quelle version est la plus récente. `_charge_le` dit quand elle est entrée : ce qui est arrivé depuis hier, et quand une anomalie a été introduite.

Théorie : `Theorie_03_DE.md`, section 4.

12. `KUNNR` et `VBELN` portent des zéros de tête (`0000200154`) qu'un entier ferait disparaître. `AEDAT` contient `00000000` pour une commande jamais modifiée, ce qui n'est pas une date. Dans la `RAW`, les deux restent en texte, telles que livrées ; le décodage appartient au `STAGING`.

Théorie : `Theorie_03_DE.md`, section 4.

13. Il devient `NULL`. C'est la convention par défaut de Snowflake et de Databricks : la `RAW` se charge de la même façon sur les trois plateformes, et l'absence de valeur se distingue d'une chaîne vide écrite volontairement.

Théorie : `Theorie_03_DE.md`, section 4.

14. Le bronze range des fichiers dans le lac, en Parquet, pour qui lit des fichiers ; la `RAW` range des tables dans l'entrepôt, pour qui écrit du SQL et du dbt. La télémétrie vit dans les deux. Les points bruts de l'historien et du compteur restent au bronze, et leurs séries nettoyées iront dans le schéma `argent` au Cours 08.

Théorie : `Theorie_03_DE.md`, section 4.

15. Il affiche « à jour » pour les 22 livraisons, puis « Total : 0 fichier(s), 0 lignes », en moins d'une seconde. Chaque fichier est déjà au journal avec la même empreinte : il n'y a rien à charger. C'est l'idempotence, vue de la ligne de commande.

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_03_DE.md`, section 3.

16. Il s'arrête avec « L'entrepôt est occupé : fermez la ligne de commande DuckDB, puis relancez. », suivi de l'erreur de DuckDB, qui nomme le programme en cause : `File is already open in …\duckdb.exe`. Le message parle de la ligne de commande, mais l'interface web est le même programme. Sous Windows, le programme qui a ouvert `boreal.duckdb` le verrouille, même s'il ne fait que lire. La règle des labos en découle : `DETACH entrepot;` dans le carnet, ou fermer l'interface, avant de lancer un script qui écrit.

Labo : `Labo_03_DE.md`, étape 2.

17. Parce qu'un document n'est pas une table : le découper en passages et l'indexer pour une recherche appartient à la formation AI Engineering, et rien ne se décide ici à sa place. Le catalogue, lui, se lit en SQL : il entre aussi dans la `RAW`, `raw.corpus_index`, avec ses 33 lignes. Le dépôt est idempotent comme le reste : 35 fichiers déposés au premier passage, 0 au second, les 35 déjà à jour.

Théorie : `Theorie_03_DE.md`, section 5.

Questions pratiques

P1.

```sql
SELECT table_raw, fichier, lignes
FROM entrepot.pipeline.chargements
WHERE lot = '2024-05'
ORDER BY table_raw;

SELECT COUNT(*) AS fichiers, SUM(lignes) AS lignes
FROM entrepot.pipeline.chargements
WHERE lot = '2024-05';
```

Résultat : 14 fichiers et 9 222 lignes. Les neuf tables de l'ERP et les cinq du portail des garanties, dont deux fichiers vides, `CDHDR.csv` et `CDPOS.csv` : aucune fiche de concessionnaire n'a changé en mai 2024. Les plus grosses livraisons du mois sont `garanties_visites` (2 199 lignes) et `erp_vbap` (1 950). Le lot de la météo, de l'usine et du corpus n'est pas `2024-05` mais `complet` : ces sources livrent un export entier, pas un mois.

Théorie : `Theorie_03_DE.md`, sections 3 et 4.

P2.

```sql
SELECT COUNT(DISTINCT _lot) AS lots, MIN(_lot) AS premier, MAX(_lot) AS dernier,
       COUNT(*) FILTER (WHERE _lot = '2025-09') AS lignes_2025_09,
       COUNT(*) AS lignes_en_tout
FROM entrepot.raw.garanties_clients;
```

Résultat : 48 lots, de `2022-01` à `2025-12`, 445 lignes au lot `2025-09`, 25 379 lignes en tout. Le portail livre un extrait par mois sur quatre ans, et chacun porte les clients créés ou modifiés dans le mois. Les lots se trient à l'alphabet parce qu'ils s'écrivent `AAAA-MM` : c'est la raison de ce format.

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_03_DE.md`, sections 2 et 4.

P3.

```
WITH etat AS (
    SELECT * FROM entrepot.raw.garanties_clients
    QUALIFY ROW_NUMBER() OVER (PARTITION BY client_id ORDER BY _lot DESC) = 1)
SELECT COUNT(*) AS clients FROM etat;

SELECT COUNT(*) AS lignes, COUNT(DISTINCT client_id) AS clients_distincts
FROM entrepot.raw.garanties_clients;
```

Résultat : 23 527 clients pour 25 379 lignes, soit 1 852 lignes de plus que de clients. L'écart, ce sont les clients revenus dans un extrait plus tardif parce que leur fiche a changé : un déménagement, un consentement retiré, un effacement demandé. La `RAW` garde toutes les versions ; `QUALIFY` avec `ROW_NUMBER()` sur le lot décroissant ne garde que la dernière de chaque client. C'est le premier geste du `STAGING` d'Analytics Engineering. La numérotation par fenêtre et son filtre, `QUALIFY`, sont au Cours 04 de la formation SQL avancé, sections 3 et 7.

Théorie : `Theorie_03_DE.md`, sections 2 et 4.

P4.

```
WITH bis AS (SELECT * EXCLUDE (_fichier, _lot, _charge_le) FROM entrepot.raw.erp_vbak WHERE _lot = '2023-03_bis'),
     mars AS (SELECT * EXCLUDE (_fichier, _lot, _charge_le) FROM entrepot.raw.erp_vbak WHERE _lot = '2023-03')
SELECT (SELECT COUNT(*) FROM bis) AS lignes_bis,
       (SELECT COUNT(*) FROM mars) AS lignes_mars,
       (SELECT COUNT(*) FROM (FROM bis EXCEPT FROM mars)) AS bis_sans_mars,
       (SELECT COUNT(*) FROM (FROM mars EXCEPT FROM bis)) AS mars_sans_bis;
```

Résultat : 609 lignes de chaque côté, 0 et 0. Les deux lots portent exactement les mêmes commandes, dans les mêmes versions : le fichier a bien été livré deux fois sous deux noms. Il faut comparer dans les deux sens, parce qu'un `EXCEPT` dans un seul sens ne verrait pas des lignes présentes seulement dans l'autre lot ; l'opérateur et le rapprochement dans les deux sens sont au Cours 07 de la formation SQL avancé, section 6. Les trois colonnes de lignage sont exclues, sans quoi chaque ligne différerait par son `_fichier` : `SELECT * EXCLUDE (…)` est au Cours 02 de la même formation, section 1.

Théorie : `Theorie_03_DE.md`, sections 3 et 4.

P5.

```
WITH par_id AS (
    SELECT message ->> 'message_id' AS message_id, COUNT(*) AS recus
    FROM entrepot.raw.telemetrie_sorties
    GROUP BY 1)
SELECT COUNT(*) FILTER (WHERE recus > 1) AS identifiants_en_double,
       SUM(recus - 1) AS messages_en_trop,
```


© 2026 iAlexMG. Tous droits réservés.

```
       MAX(recus) AS recus_max
FROM par_id;
```

Résultat : 2 303 identifiants reçus deux fois, 2 303 messages en trop, et aucun reçu plus de deux fois. On retrouve bien les 2 303 doublons du générateur, et le compte de la table le confirme : 158 146 messages pour 155 843 identifiants distincts. C'est la livraison « au moins une fois » de MQTT : le véhicule a republié un message dont il n'a pas reçu l'accusé de réception. La `RAW` les garde, et c'est au consommateur de dédoublonner sur `message_id`.

Théorie : `Theorie_03_DE.md`, sections 3 et 4.

P6.

```
SELECT table_raw, SUM(lignes) AS lignes, COUNT(*) AS fichiers
FROM entrepot.pipeline.chargements
GROUP BY table_raw
ORDER BY lignes DESC
LIMIT 5;
```

Résultat : `telemetrie_sorties` 158 146 lignes en 884 fichiers, `erp_vbap` 81 292 en 49, `garanties_visites` 53 865 en 48, `meteo_horaire` 37 068 en un seul fichier, et `garanties_consentements` 32 708 en 48. Le journal répond sans lire une seule table de la `RAW` : il porte le compte de lignes de chaque chargement. Additionner les lignes du journal donne le même total que compter les tables, tant qu'aucun fichier n'a été rechargé sous une autre empreinte.

Théorie : `Theorie_03_DE.md`, section 3.

Diagnostics

E1. Le chargement n'est pas idempotent : sans journal, chaque relance ajoute les mêmes lignes. 53 076, c'est exactement deux fois 26 538, le compte d'un chargement complet des extraits : tout a été chargé deux fois. Les rapports doublent le chiffre d'affaires parce qu'ils additionnent deux fois chaque commande. Le correctif tient en deux gestes. D'abord, vider la table et recharger avec un journal des chargements, qui reconnaît un fichier par son chemin et son empreinte, et qui s'écrit dans la même transaction que les données. Ensuite, dire au collègue que 26 538 n'est pas non plus le bon chiffre pour un rapport : les extraits sont incrémentaux, et l'état courant se tire en gardant la version du lot le plus récent de chaque commande, soit 25 483.

Théorie : `Theorie_03_DE.md`, sections 2 et 3.

E2. Reconnaître un fichier à sa seule empreinte casse les livraisons complètes identiques. Les 49 fichiers `T005U` des extraits ont exactement le même contenu, mois après mois, parce que la table des régions ne change jamais : seul le premier est chargé, et la `RAW` n'a plus de régions après le lot `2022-01`. L'ingénieur analytique, qui prend le dernier lot pour obtenir l'état courant, ne trouve rien. Le correctif est de revenir à la clé du chargeur, le chemin, avec l'empreinte comme témoin de changement. Le doublon de `2023-03_bis` n'est pas l'affaire du chargeur : il est livré sous un autre nom, donc c'est une autre livraison, et c'est un contrat de données qui le signale (Cours 11).

Théorie : `Theorie_03_DE.md`, section 3.

Questions à développer

© 2026 iAlexMG. Tous droits réservés.

A. Ce que la `RAW` garantit, c'est de rendre ce que les sources ont livré, tel qu'elles l'ont livré. Retirer les 609 commandes de `2023-03_bis` reviendrait à décider, dans la couche brute, laquelle des deux livraisons compte : c'est une règle métier, et elle n'a rien à faire là. On y perdrait aussi la preuve de ce qui est arrivé, la trace du problème de l'ERP, et le test d'unicité de l'ingénieur analytique n'aurait plus rien à prouver. Ce qu'on répond au responsable des ventes : le doublon est visible et nommé, marqué par son lot ; aucun rapport ne le compte deux fois, parce que le `STAGING` d'Analytics Engineering garde la version du lot le plus récent de chaque commande ; un contrat de données le signale à chaque chargement (Cours 11) ; et la vraie correction, celle qui vaut la peine, se fait chez l'ERP, qui ne doit plus livrer deux fois le même mois sous deux noms.

Théorie : `Theorie_03_DE.md`, sections 3 et 4.

Mises en situation

M1. Coopérative forestière des Hautes-Terres.

Choix	Décision	Pourquoi
Clé du journal	le chemin du fichier, avec son empreinte comme témoin de changement	le redépôt sous le même nom se voit à l'empreinte ; le dépôt sous un autre nom reste une livraison distincte
Règle du lot	la date tirée du nom de fichier, et la scierie en colonne à part	les analystes filtrent par journée, et chaque scierie se compte séparément
Ce qui entre dans la `RAW`	tout, en texte, avec `_fichier`, `_lot` et `_charge_le`	la couche brute ne tranche pas entre deux versions d'une journée
Contrôle	deux fichiers d'une même scierie pour une même date, ou deux empreintes identiques sous deux noms	c'est exactement ce que le journal permet de voir

Le fichier corrigé, redéposé sous le même nom, est reconnu par son chemin avec une autre empreinte : le chargeur retire les lignes de la première version, charge la seconde, et le journal garde les deux passages avec leurs empreintes et leurs heures. C'est là que la direction lit ce qui a été corrigé, et quand. Le fichier redéposé sous un suffixe `_v2` est, pour le journal, une livraison de plus : ses lignes s'ajoutent, marquées par leur `_fichier`, et c'est le contrôle qui le signale. Serait acceptable aussi : faire du couple scierie et date la clé du journal, ce qui attrape le `_v2` tout seul, au prix d'une règle de plus dans le chargeur. Serait indéfendable : supprimer les lignes de la veille à chaque dépôt, qui efface la preuve du redépôt.

Théorie : `Theorie_03_DE.md`, sections 3 et 4.

© 2026 iAlexMG. Tous droits réservés.